Tema 9

Control y manejo de excepciones

Programación

Tech Hub <FP>
euroformac

ÍNDICE

Objetivos de la unidad 3
1. Introducción 4
2. Excepciones. Concepto 5
2.1 Concepto 5
3. Jerarquías de excepciones 7
3.1 Por qué usar jerarquías de excepciones 7
3.2 ¿Cómo funcionan las jerarquías de excepciones? 7
3.3 ¿Cómo manejar excepciones en una jerarquía de excepciones? 8
4. Manejo de excepciones 9
4.1 Captura de excepciones 11
4.2 Propagar excepciones 13
4.3 Lanzar excepciones 15
4.4 Crear clases de excepciones 17
5. Resumen 20
Glosario 21
Bibliografía 22

Objetivos de la unidad

- Comprender el concepto de excepciones y su importancia en la programación.
- Aprender a reconocer y manejar excepciones de manera efectiva en tus programas.
- Conocer la jerarquía de excepciones y cómo utilizarla para capturar errores de manera más específica.
- Aprender a capturar, propagar y lanzar excepciones.
- Crear tus propias clases de excepciones personalizadas para manejar errores específicos en tus programas.
- Implementar un manejo adecuado de excepciones en tus programas para hacerlos más robustos, seguros y fáciles de mantener.

Tech Hub <FP> euroformac

1. Introducción

En esta unidad aprenderás acerca de las **excepciones** en la programación y cómo manejarlas adecuadamente en tus programas. Las excepciones son errores que ocurren durante la ejecución de un programa y que pueden detener su ejecución abruptamente. Aprenderás a reconocer y manejar estas excepciones para evitar que el programa termine de manera inesperada y puedas tomar decisiones en caso de que se presente un error.

También aprenderás acerca de la **jerarquía de excepciones** y cómo puedes utilizarlas para organizar tus excepciones y capturarlas de manera más efectiva. Aprenderás a capturar y propagar excepciones, lanzar excepciones manualmente cuando sea necesario y crear tus propias clases de excepciones personalizadas para manejar errores específicos.

Al finalizar esta unidad, podrás implementar un manejo adecuado de excepciones en tus programas, lo que hará que sean más robustos, seguros y fáciles de mantener.

Tech Hub <FP> euroformac

2. Excepciones. Concepto

En programación una **excepción** es un evento o situación que interrumpe el flujo normal de ejecución de un programa. Estas situaciones pueden ser provocadas por errores en el código, por eventos externos impredecibles o por problemas de *hardware* o de red, entre otros.

El manejo de excepciones es una técnica fundamental para garantizar la estabilidad y fiabilidad de los programas. Cuando una excepción se produce, el programa debe ser capaz de detectarla, analizarla y responder de manera adecuada para evitar que se produzcan daños mayores.

Por lo tanto, conocer cómo manejar las excepciones es una habilidad clave para cualquier programador, ya que le permite anticiparse a los posibles errores y diseñar programas más robustos y resistentes a fallos.

2.1 Concepto

Las e**xcepciones** son un mecanismo que permite a los programadores manejar errores y situaciones inesperadas en sus programas de manera controlada. Cuando un error ocurre en un programa se lanza una excepción, lo que interrumpe el flujo normal de la ejecución y permite que el programador tome medidas para corregir el error o manejarlo de manera adecuada.

Por ejemplo, supongamos que tienes un programa que lee un archivo y lo procesa. Si el archivo no existe o no se puede leer por alguna razón, el programa podría lanzar una excepción para indicar que ha ocurrido un error. En lugar de simplemente detenerse, el programa podría capturar la excepción y mostrar un mensaje de error al usuario, o intentar recuperarse de alguna otra manera.

Existen **dos tipos de excepciones**: **excepciones verificadas** y **excepciones no verificadas**. Las excepciones verificadas son aquellas que son parte de la firma de un método y que el programador debe capturar o propagar, mientras que las excepciones no verificadas son aquellas que pueden ocurrir en cualquier momento y que el programador no está obligado a capturar o propagar.

En resumen, las excepciones son una herramienta esencial para cualquier programador, ya que permiten manejar errores y situaciones inesperadas de manera controlada. Es importante conocer los diferentes tipos de excepciones

y saber cómo manejarlos adecuadamente para escribir programas más confiables y seguros.

Definición

Una **excepción** es un evento que se produce durante la ejecución de un programa y que interrumpe su flujo normal. Puede ser causada por errores en el código, entradas incorrectas del usuario o problemas con los recursos del sistema.

Más info

Para una explicación más detallada sobre excepciones y su manejo, se puede consultar el capítulo 11 del libro: Schildt, H. (2018). Java: The Complete Reference, Eleventh Edition. McGraw-Hill Education.

3. Jerarquías de excepciones

Las **jerarquías de excepciones** son una parte fundamental de la programación orientada a objetos que permite manejar de manera adecuada situaciones imprevistas o errores en la ejecución de un programa. En esencia, una excepción es una señal de que algo ha salido mal en la ejecución del código y, por lo tanto, el programa no puede continuar normalmente. Las jerarquías de excepciones son una forma de clasificar y organizar las diferentes excepciones que pueden ocurrir en un programa para poder manejarlas de manera eficiente y efectiva. En esta asignatura se explorará en detalle cómo se definen y se utilizan las jerarquías de excepciones en diferentes lenguajes de programación.

3.1 Por qué usar jerarquías de excepciones

Las jerarquías de excepciones son útiles para organizar las excepciones en grupos relacionados y definir cómo se deben manejar. En lugar de tener que manejar cada excepción individualmente, las jerarquías de excepciones permiten definir reglas generales para un conjunto de excepciones relacionadas.

3.2 ¿Cómo funcionan las jerarquías de excepciones?

Las jerarquías de excepciones funcionan mediante la creación de una estructura **jerárquica de clases de excepción.** Cada clase de excepción representa un tipo específico de excepción. Las clases de excepción se organizan en una estructura jerárquica, de modo que las excepciones más específicas estén en la parte inferior de la jerarquía y las excepciones más generales estén en la parte superior.

Ejemplo de jerarquía de excepciones

Un ejemplo de jerarquía de excepciones podría ser la siguiente:

- **Exception** (la clase base para todas las excepciones)
  - **IOException** (excepciones relacionadas con la entrada/salida)
    - FileNotFoundException (el archivo solicitado no se encuentra)
    - EOFException (fin de archivo inesperado)
  - **RuntimeException** (excepciones relacionadas con errores de programación)

- NullPointerException (se intenta acceder a una variable nula)
- ArrayIndexOutOfBoundsException (se intenta acceder a una posición de un array que no existe)

3.3 ¿Cómo manejar excepciones en una jerarquía de excepciones?

Para manejar excepciones en una jerarquía de excepciones, se pueden utilizar las estructuras **try-catch** y **finally**. En el bloque **try** se coloca el código que se ejecutará y en caso de producirse una excepción, se capturará en el bloque **catch** correspondiente. En el bloque **finally** se coloca el código que se ejecutará independientemente de si se ha producido una excepción o no.

Las **jerarquías de excepciones** son una herramienta importante para manejar excepciones de manera efectiva y eficiente en programas. Al organizar las excepciones en grupos relacionados y definir cómo se deben manejar, las jerarquías de excepciones pueden simplificar el proceso de manejo de excepciones y hacer que los programas sean más robustos y seguros.

Definición

Las **jerarquías de excepciones** son un sistema de clasificación que agrupa las excepciones en categorías según su tipo y gravedad. En Java, todas las excepciones son subclases de la clase "Throwable".

Más info

Para una comprensión más profunda sobre las jerarquías de excepciones, se puede revisar la documentación oficial de Java en el siguiente enlace:

https://docs.oracle.com/en/java/javase/11/docs/api/java.base/java/lang/Throwable.html

4. Manejo de excepciones

Las **excepciones** son eventos que ocurren durante la ejecución de un programa que interrumpen el flujo normal de la ejecución. Pueden ser causadas por errores de programación, errores en la entrada del usuario, problemas de *hardware*, etc. El manejo de excepciones se refiere a la forma en que un programa maneja estos eventos inesperados.

Las excepciones en muchos lenguajes de programación se organizan en jerarquías. La jerarquía de excepciones proporciona una estructura de clases que representa diferentes tipos de excepciones. La clase base en la jerarquía de excepciones generalmente se llama "**Exception**" y todas las demás clases de excepción derivan de ella.

Cuando una excepción ocurre durante la ejecución del programa, el sistema busca la clase de excepción más específica que pueda manejar la excepción. Si no se encuentra una clase de excepción adecuada, el programa se detiene y se muestra un mensaje de error al usuario.
Por ejemplo, en el lenguaje de programación Java, la clase "**Exception**" es la clase base de la jerarquía de excepciones. A continuación, hay algunas de las clases de excepción más comunes en Java:

- **ArithmeticException**: Se lanza cuando ocurre un error aritmético, como la división por cero.
- **NullPointerException**: Se lanza cuando se intenta utilizar un objeto nulo.
- **IndexOutOfBoundsException:** Se lanza cuando se intenta acceder a un índice fuera de los límites de un arreglo o una lista.
- **FileNotFoundException:** Se lanza cuando se intenta acceder a un archivo que no existe.
- **IOException:** Se lanza cuando ocurre un error durante la entrada o salida de datos.

Para manejar una excepción en un programa, se utiliza una estructura **try-catch**. El código que puede lanzar una excepción se coloca dentro de un bloque **try**. Si ocurre una excepción dentro del bloque try, el programa busca un bloque **catch** correspondiente que pueda manejar la excepción. Si se encuentra un bloque catch adecuado, el código dentro del bloque catch se ejecuta.

Por ejemplo, en **Java**, podríamos tener el siguiente código para manejar la excepción de división por cero:

```
try {
    int resultado = 10 / 0;
    } catch (ArithmeticException e) {
    System.out.println("Error: división por cero");
    }
```

En este ejemplo, el código que realiza la división se encuentra dentro del bloque try. Si se produce una excepción de división por cero, el programa busca un bloque catch que pueda manejar la excepción. En este caso, el bloque catch correspondiente es el que maneja la excepción **ArithmeticException**. El mensaje "Error: división por cero" se muestra en la pantalla si ocurre la excepción.

A continuación, un ejemplo en **Python** sería el siguiente:

```
    try:
    num1 = int(input("Ingrese el primer número: "))
    num2 = int(input("Ingrese el segundo número: "))
    resultado = num1 / num2
    print("El resultado de la división es:", resultado)
except ValueError:
    print("Error: Ingrese solo números enteros")
except ZeroDivisionError:
    print("Error: No se puede dividir entre cero")
except Exception as e:
    print("Error desconocido:", e)
```

En este ejemplo, se está realizando una división entre dos números ingresados por el usuario. Sin embargo, pueden ocurrir diferentes tipos de excepciones, como un error al ingresar los números o una división por cero. Para manejar estas excepciones, se utiliza la estructura try-catch.

En el bloque try, se realiza la división y se imprime el resultado. Si ocurre una excepción, el programa busca un bloque catch correspondiente que pueda manejar la excepción. En este caso, se han agregado tres bloques catch:

- **El primer bloque catch** maneja la excepción **ValueError,** que ocurre si el usuario ingresa un valor que no es un número entero.
- **El segundo bloque catch** maneja la excepción **ZeroDivisionError,** que ocurre si el usuario intenta dividir entre cero.
- **El tercer bloque catch** maneja cualquier otra excepción no prevista y muestra un mensaje de error genérico con información adicional sobre el error.

De esta manera, el programa maneja de forma adecuada las excepciones que pueden ocurrir durante la ejecución del programa y evita que el programa se detenga inesperadamente.

4.1 Captura de excepciones

La **captura de excepciones** se refiere a la forma en que un programa maneja las excepciones que ocurren durante la ejecución del mismo. Cuando se produce una excepción, el programa puede detenerse o mostrar un mensaje de error al usuario. Sin embargo, es posible capturar la excepción y tomar medidas para manejarla de forma adecuada, evitando que el programa se detenga inesperadamente.
Las excepciones en muchos lenguajes de programación se organizan en jerarquías. La jerarquía de excepciones proporciona una estructura de clases que representa diferentes tipos de excepciones. La clase base en la jerarquía de excepciones generalmente se llama "**Exception**" y todas las demás clases de excepción derivan de ella.

Cuando una excepción ocurre durante la ejecución del programa, el sistema busca la clase de excepción más específica que pueda manejar la excepción. Si no se encuentra una clase de excepción adecuada, el programa se detiene y se muestra un mensaje de error al usuario.

Para capturar una excepción, se utiliza una estructura **try-catch**. El código que puede lanzar una excepción se coloca dentro de un bloque try. Si ocurre una excepción dentro del bloque try, el programa busca un bloque catch correspondiente que pueda manejar la excepción. Si se encuentra un bloque catch adecuado, el código dentro del bloque catch se ejecuta.

A continuación, se presenta un ejemplo de captura de excepciones en **Java**. Supongamos que queremos leer un archivo y mostrar su contenido en la pantalla. Si el archivo no existe, se produce una excepción **FileNotFoundException.** Para manejar esta excepción, podemos usar la estructura try-catch de la siguiente manera:

```
try {
    FileReader fr = new FileReader("archivo.txt");
    BufferedReader br = new BufferedReader(fr);

    String linea;
    while ((linea = br.readLine()) != null) {
        System.out.println(linea);
    }

    br.close();
    } catch (FileNotFoundException e) {
    System.out.println("Error: archivo no encontrado");
    } catch (IOException e) {
    System.out.println("Error al leer el archivo");
    }
```

En este ejemplo, el código que lee el archivo se encuentra dentro del bloque try. Si se produce una excepción, el programa busca un bloque catch correspondiente que pueda manejar la excepción. En este caso, se han agregado dos bloques catch:

- **El primer bloque catch** maneja la excepción **FileNotFoundException**, que ocurre si el archivo que intentamos leer no existe.
- **El segundo bloque catch** maneja la excepción I**OException**, que puede ocurrir si se produce un error durante la lectura del archivo.

De esta forma, el programa captura las excepciones que pueden ocurrir durante la lectura del archivo y muestra un mensaje de error adecuado para cada una de ellas.

Importante

La captura de excepciones es el proceso de detectar y manejar las excepciones que se producen durante la ejecución de un programa.

Sabías que

En Java, la cláusula "try" se utiliza para encapsular el código que puede generar una excepción, mientras que la cláusula "catch" se utiliza para manejar las excepciones capturadas.

4.2 Propagar excepciones

Propagar excepciones se refiere a la forma en que un programa pasa una excepción de un método a otro. Cuando se produce una excepción en un método, el programa puede detenerse o mostrar un mensaje de error al usuario. Sin embargo, es posible propagar la excepción y enviarla al método que lo llamó, permitiendo que ese método maneje la excepción de forma adecuada.

Las excepciones en muchos lenguajes de programación se organizan en jerarquías. La jerarquía de excepciones proporciona una estructura de clases que representa diferentes tipos de excepciones. La clase base en la jerarquía de excepciones generalmente se llama "**Exception**" y todas las demás clases de excepción derivan de ella.

Cuando se produce una excepción en un método, el sistema busca la clase de excepción más específica que pueda manejar la excepción. Si no se encuentra una clase de excepción adecuada, el programa se detiene y se muestra un mensaje de error al usuario. Sin embargo, si el método no maneja la excepción, puede propagarla a su método llamador usando la palabra clave "**throws**".

La palabra clave "**throws**" se usa para declarar que un método puede lanzar una excepción específica. Si un método lanza una excepción y no la maneja, debe declarar que la excepción puede ser lanzada utilizando la palabra clave "**throws**". Cuando se llama al método, el programa verifica si se lanza alguna excepción. Si se lanza una excepción, el programa busca el método llamador que maneja la excepción.

A continuación, se presenta un ejemplo de propagación de excepciones en **Java**. Supongamos que tenemos dos métodos, uno que abre un archivo y otro que lee el contenido del archivo. Si el archivo no existe, el método de apertura de archivo lanza una excepción **FileNotFoundException**. En este caso, queremos propagar la excepción al método llamador y manejarla allí.

```
import java.io.*;


public class Archivo {


  public static void main(String[] args) {
    try {
      leerArchivo("archivo.txt");
    } catch (FileNotFoundException e) {
      System.out.println("Error: archivo no encontrado");
    }
  }


  public static void leerArchivo(String nombreArchivo) throws
FileNotFoundException {
    FileReader fr = new FileReader(nombreArchivo);
    BufferedReader br = new BufferedReader(fr);


    String linea;
    try {
      while ((linea = br.readLine()) != null) {
        System.out.println(linea);
      }
    } catch (IOException e) {
      System.out.println("Error al leer el archivo");
    } finally {
      try {
        br.close();
      } catch (IOException e) {
```

```
        System.out.println("Error al cerrar el archivo");
      }
    }
  }

}
```

En este ejemplo, el método "**leerArchivo**" abre el archivo y lee su contenido. Si ocurre una excepción al abrir el archivo, el método lanza una excepción **FileNotFoundException** utilizando la palabra clave "**throws**". El método llamado, "**main**", maneja la excepción utilizando un bloque try-catch.

De esta forma, el programa propaga la excepción **FileNotFoundException** desde el método "leerArchivo" al método "main", permitiendo que este último maneje la excepción de forma adecuada.

Importante

Propagar una excepción significa pasarla a una función superior en la pila de llamadas para que sea manejada en otro lugar.

Sabías que

En Java, se utiliza la palabra clave "throws" para indicar que una función puede lanzar una excepción y pasarla al llamante.

4.3 Lanzar excepciones

Lanzar excepciones se refiere a la forma en que un programa indica que se ha producido un error o una excepción. En lugar de detener el programa o mostrar un mensaje de error al usuario, el programa puede lanzar una excepción que indique el problema y permita que el código que llama al método maneje la excepción de forma adecuada.

Cuando se lanza una excepción, el programa crea una instancia de la clase de excepción correspondiente y la "lanza" hacia arriba en la jerarquía de llamadas

de método. El método que llama puede manejar la excepción usando un bloque try-catch o propagar la excepción a su método llamador.

A continuación, se presenta un ejemplo de lanzamiento de excepciones en **Java**. Supongamos que tenemos un método que realiza una división entre dos números. Si el segundo número es cero, se produce una excepción "**ArithmeticException**". En este caso, queremos lanzar la excepción para indicar que se ha producido un error en la división.

```
public class Division {


    public static void main(String[] args) {
      int a = 10;
      int b = 0;


      try {
        int resultado = dividir(a, b);
        System.out.println("El resultado es: " + resultado);
      } catch (ArithmeticException e) {
        System.out.println("Error: división entre cero");
      }
    }


    public static int dividir(int a, int b) throws ArithmeticException {
      if (b == 0) {
        throw new ArithmeticException("División entre cero");
      }
      return a / b;
    }


  }
```

En este ejemplo, el método "dividir" comprueba si el segundo número es cero y, si lo es, lanza una excepción ArithmeticException utilizando la palabra clave "throw". El método llamador, "main", maneja la excepción utilizando un bloque try-catch.

De esta forma, el programa lanza la excepción ArithmeticException cuando se intenta dividir entre cero, indicando que se ha producido un error en la división.

Importante

Es importante destacar el manejo de excepciones en la programación. No manejar adecuadamente las excepciones puede provocar errores y fallos en la aplicación, lo que puede afectar negativamente la experiencia del usuario.
Lanzar una excepción significa crear una instancia de una clase de excepción y enviarla al llamante de la función actual para que la maneje.

Sabías que

En Java, se utiliza la palabra clave "throw" para lanzar una excepción en una función.

4.4 Crear clases de excepciones

En la **programación orientada a objetos** las excepciones también se pueden representar mediante **clases**. Para crear una **clase de excepción personalizada** se debe extender la clase de excepción base correspondiente en la jerarquía de excepciones. Esto nos permite crear nuestras propias clases de excepción para manejar situaciones específicas en nuestro programa.

A continuación, se presenta un ejemplo de cómo crear una clase de excepción personalizada en **Java**. Supongamos que tenemos una clase que representa una cuenta bancaria. Si se intenta retirar más dinero del que se tiene disponible en la cuenta, queremos lanzar una excepción personalizada "**FondosInsuficientesException**".

```
public class CuentaBancaria {

    private double saldo;

    public CuentaBancaria(double saldoInicial) {
```

```
      this.saldo = saldoInicial;
    }

    public void retirar(double cantidad) throws
FondosInsuficientesException {
      if (cantidad > saldo) {
        throw new FondosInsuficientesException("Saldo insuficiente");
      }
      saldo -= cantidad;
      System.out.println("Retirada exitosa: " + cantidad + "€");
    }

    public double getSaldo() {
      return saldo;
    }

    public static void main(String[] args) {
      CuentaBancaria cuenta = new CuentaBancaria(100.0);
      try {
        cuenta.retirar(150.0);
      } catch (FondosInsuficientesException e) {
        System.out.println("Error: " + e.getMessage());
      }
      System.out.println("Saldo actual: " + cuenta.getSaldo() + "€");
    }

  }
  class FondosInsuficientesException extends Exception {

    public FondosInsuficientesException(String mensaje) {
      super(mensaje);
    }
```

```
}
```

En este ejemplo, creamos la clase de excepción personalizada "FondosInsuficientesException" extendiendo la clase "**Exception**". La clase tiene un constructor que acepta un mensaje de error y lo pasa a la clase base usando la palabra clave "super".

Dentro del método "retirar" de la clase "CuentaBancaria", comprobamos si la cantidad solicitada es mayor que el saldo disponible. Si es así, lanzamos la excepción personalizada "FondosInsuficientesException" utilizando la palabra clave "**throw**".

En el método "**main**" manejamos la excepción "FondosInsuficientesException" utilizando un bloque **try-catch**. Si se produce la excepción imprimimos el mensaje de error. De lo contrario, el programa continúa ejecutando y muestra el saldo actual de la cuenta bancaria.

De esta forma, la clase de excepción personalizada "FondosInsuficientesException" nos permite manejar situaciones específicas en nuestro programa, como cuando no hay suficiente saldo en una cuenta bancaria para realizar una retirada.

Importante

El manejo de excepciones es una técnica fundamental en la programación moderna, ya que permite identificar y solucionar errores de forma más eficiente.

Sabías que

En Java, el manejo de excepciones se realiza a través de las cláusulas "try", "catch" y "finally".

5. Resumen

Esta es una de las unidades fundamentales en programación que permite aprender cómo manejar y controlar errores en un código. Durante la unidad se han recogido cómo identificar y clasificar los diferentes tipos de excepciones que pueden surgir durante la ejecución de un programa. Además del uso de estructuras try-catch para capturar y manejar las excepciones que pueden surgir y a utilizar el throw y throws para lanzar y propagar errores en el código.

Además, se ha indicado cómo crear excepciones personalizadas para manejar situaciones específicas en el código. Se ha hecho alusión a el uso de excepciones para proporcionar información útil al usuario, lo que puede mejorar significativamente la calidad de la experiencia del usuario y la facilidad de mantenimiento del programa.

En resumen, en esta unidad se ha proporcionado las herramientas necesarias para controlar y solucionar errores en el código de un programa. Al aprender a manejar las excepciones, se puede escribir programas más robustos y confiables y evitar errores comunes que pueden surgir durante la ejecución del código. Esto permite construir software más eficiente y de calidad, lo que es crucial en un entorno cada vez más competitivo y en constante evolución.

Glosario

- **Excepción**: evento anormal que ocurre durante la ejecución de un programa y que interrumpe su flujo normal.
- **Clase de excepción**: tipo de excepción definido por el usuario que permite manejar situaciones específicas.
- **Bloque try-catch:** estructura de control que permite capturar excepciones y manejarlas de manera adecuada.
- **Bloque finally:** bloque de código que se ejecuta siempre, independientemente de si se ha producido una excepción o no.
- **Lanzar una excepción:** proceso mediante el cual se indica que se ha producido una excepción en el programa.
- **Propagar una excepción:** proceso mediante el cual se pasa la responsabilidad de manejar una excepción a un método que lo llame.
- **Jerarquía de excepciones:** estructura de clases que establece una relación de herencia entre las distintas excepciones, permitiendo su agrupación por categorías.
- **Checked exceptions:** excepciones que obligan al programador a manejarlas explícitamente, ya sea mediante un bloque try-catch o lanzándolas en la firma del método.
- **Unchecked exceptions:** excepciones que no necesitan ser manejadas explícitamente por el programador.
- **Stack trace:** información que proporciona la pila de llamadas del programa cuando se produce una excepción, indicando el lugar exacto donde se produjo el error.

Bibliografía

"Java Concurrency in Practice" de Brian Goetz, Tim Peierls, Joshua Bloch, Joseph Bowbeer, David Holmes, and Doug Lea

Documentación oficial de Python: Python Software Foundation. (s.f.). Built-in Exceptions:
https://docs.python.org/3/library/exceptions.html

Sweigart, A. (2019). Automate the Boring Stuff with Python, 2nd Edition: Practical Programming for Total Beginners. No Starch Press.

Schildt, H. (2018). Java: The Complete Reference, Eleventh Edition. McGraw-Hill Education.

Beazley, D. M., & Jones, B. K. (2013). Python Essential Reference (4th ed.). Addison-Wesley Professional.